



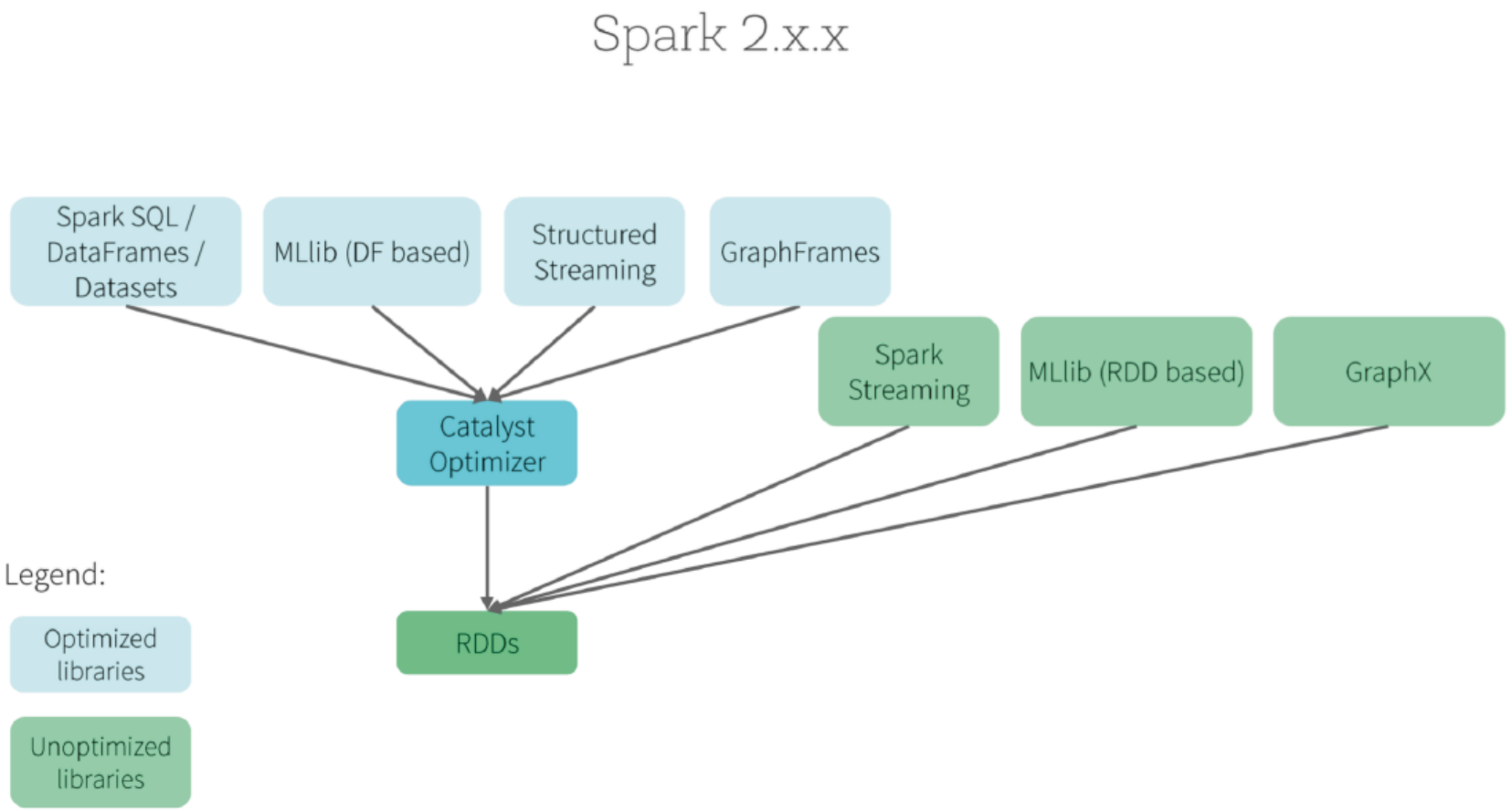
Tema 3: Librerías/Componentes de Spark

Rafael Pastor Vargas
Andrés Duque Fernández
SCC-ETSI Informática-UNED

LIBRERÍAS/COMPONENTES DE SPARK

- Librerías disponibles en Spark.
- **Gestión de datos al modo SQL: Spark SQL.**
- Machine Learning en Spark: Spark MLlib.
- Procesamiento de gráfos: Spark GraphX.
- Introducción a Spark Streaming.

LIBRERIAS


Spark 2.x.x
Spark SQL / DataFrames / Datasets
MLlib (DF based)
Structured Streaming
GraphFrames
Spark Streaming
MLlib (RDD based)
GraphX
Catalyst Optimizer
RDDs
Legend:
Optimized libraries
Unoptimized libraries

SPARK SQL

- Permite la ejecución de consultas relacionales
  - Mejora las capacidades de fuentes de datos (Data Source API) de Spark (entrada de datos)
- Soporta
  - SQL
  - HiveQL
  - Scala
- SchemaRDD (antes de 2.x.x)
  - Objetos fila (Row)
  - Esquema de datos (Schema)
  - Se pueden crear desde:
    - RDDs ya creados
    - Ficheros Parquet
    - Datos JSON
    - Utilizando HiveQL (consultas sobre el datastore en Hive)

DATOS A EMPLEAR EN LOS EJEMPLOS

- NorthWind Dataset (disponible en el curso virtual)

SQLCONTEXT

- SQLContext es el punto de entrada para la creación de SchemaRDDs (antes de 2.x.x)
- Reemplazado por SparkSession (2.x.x)

Scala:

```
val sc: SparkContext // An existing SparkContext.
val sqlContext = new org.apache.spark.sql.SQLContext(sc)
```

Java:

```
JavaSparkContext sc = ...; // An existing JavaSparkContext.
JavaSQLContext sqlContext = new
org.apache.spark.sql.api.java.JavaSQLContext(sc);
```

Python:

```
from pyspark.sql import SQLContext sqlContext = SQLContext(sc)
```

SPARK SQL: EJEMPLO (1/2)

- Cargamos los datos en un Dataset (Scala)

```
val filePath = ".//fdps-v3-master//"

println(s"Running Spark Version ${sc.version}")

case class Employee(EmployeeID : String,    LastName : String, FirstName : String, Title : String,   BirthDate : String, HireDate : String,   City : String, State : String, Zip : String,  Country : String,   ReportsTo : String)

val employees = spark.read.option("header","true").csv(filePath + "data//NW-Employees.csv").as[Employee]

println("Employees has "+employees.count()+" rows")

employees.show(5)

employees.head()
```

scala> **:load** <PATH>sparkSQL.scala

SPARK SQL: EJEMPLO (2/2)

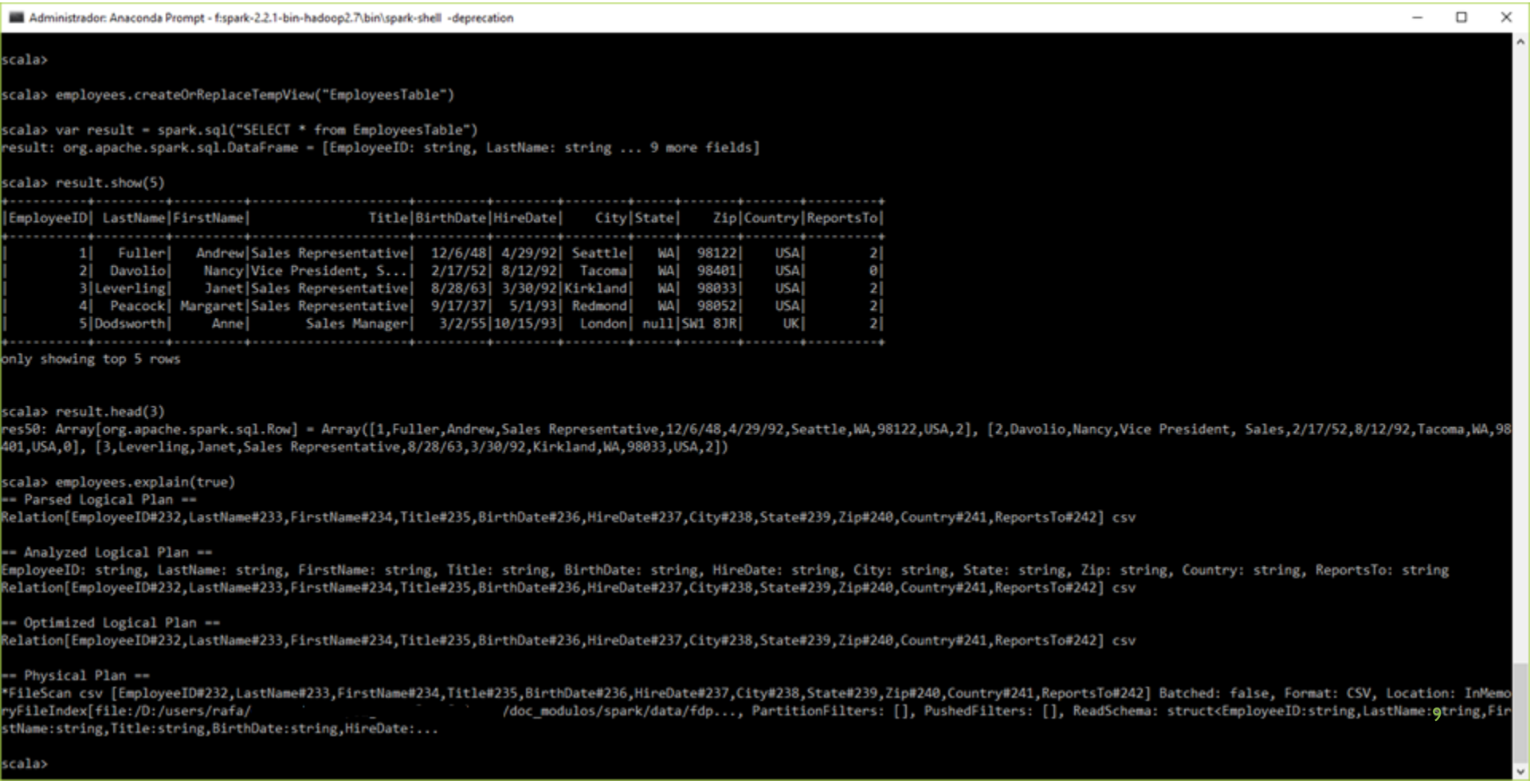
- Consultas SQL

```
employees.createOrReplaceTempView("EmployeesTable")
var result = spark.sql("SELECT * from EmployeesTable")
result.show(5)
result.head(3)
employees.explain(true)

result = spark.sql("SELECT * from EmployeesTable WHERE State = 'WA' ")
result.show(5)
result.head(3)
result.explain(true)
```

createOrReplaceTempView vs. **createTempView**

SPARK SQL: MÉTODO EXPLAIN()



SPARK SQL: INFERENCIA DE TIPOS

- Inferencia de tipos

```
val orders = spark.read.option("header","true").csv(filePath + "data/NW-Orders.csv")
println("Orders has "+orders.count()+" rows")
orders.show(5)
orders.head()
orders.dtypes
```

```
scala> orders.dtypes
res67: Array[(String, String)] = Array((OrderID,StringType), (CustomerID,StringType), (EmployeeID,StringType), (OrderDate,StringType), (ShipCountry,StringType))
```

```
val orders = spark.read.option("header","true").option("inferSchema","true").csv(filePath
+ "data/NW-Orders.csv")
println("Orders has "+orders.count()+" rows")
orders.show(5)
orders.head()
orders.dtypes
```

case class Order(OrderID : Integer, CustomerID : String, EmployeeID : Integer, OrderDate : String, ShipCountry : String)

→ .as[Order]


```
scala> orders.dtypes
res66: Array[(String, String)] = Array((OrderID,IntegerType), (CustomerID,StringType), (EmployeeID,IntegerType), (OrderDate,StringType), (ShipCountry,StringType))
```

SPARK SQL: OPERACIONES COMPLEJAS (1/2)

case class OrderDetails(OrderID : Integer, ProductID : Integer, UnitPrice : Double, Qty : Integer, Discount : Double)

- Varias tablas → Operaciones de agrupamiento, cálculo, etc. con SQL

```
val orderDetails = spark.read.option("header","true").option("inferSchema","true").csv(filePath + "data/NW-Order-Details.csv").as[OrderDetails]

orders.createOrReplaceTempView("OrdersTable")

orderDetails.createOrReplaceTempView("OrderDetailsTable")

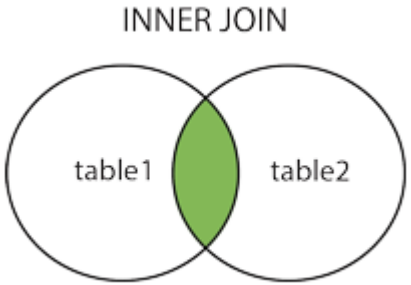
# INNER JOIN

val result = spark.sql("SELECT OrderDetailsTable.OrderID,ShipCountry,UnitPrice,Qty,Discount FROM OrdersTable INNER JOIN OrderDetailsTable ON OrdersTable.OrderID = OrderDetailsTable.OrderID")

result.show(10)

result.head(3)

result.explain(true)
```



SPARK SQL: OPERACIONES COMPLEJAS (2/2)

- Calcular las ventas por país

```
val result = spark.sql("SELECT ShipCountry,
SUM(OrderDetailsTable.UnitPrice * Qty * Discount) AS
ProductSales FROM OrdersTable INNER JOIN OrderDetailsTable
ON OrdersTable.OrderID = OrderDetailsTable.OrderID GROUP
BY ShipCountry")
```

SPARK SQL: RESULTADOS DE CONSULTAS COMPLEJAS

OrderID	ShipCountry	UnitPrice	Qty	Discount
10248	France	34.8	5	0.0
10248	France	9.8	10	0.0
10248	France	14.0	12	0.0
10249	Germany	42.4	40	0.0
10249	Germany	18.6	9	0.0
10250	Brazil	16.8	15	0.15
10250	Brazil	42.4	35	0.15
10250	Brazil	7.7	10	0.0
10251	France	16.8	20	0.0
10251	France	15.6	15	0.05
10251	France	16.8	6	0.05
10252	Belgium	27.2	40	0.0
10252	Belgium	2.0	25	0.05
10252	Belgium	64.8	40	0.05
10253	Brazil	16.0	40	0.0
10253	Brazil	14.4	42	0.0
10253	Brazil	10.0	20	0.0
10254	Switzerland	8.0	21	0.0
10254	Switzerland	19.2	21	0.15
10254	Switzerland	3.6	15	0.15

only showing top 20 rows

ShipCountry	ProductSales
Sweden	5028.5599999999995
Germany	14355.9965
France	4140.437499999999
Argentina	0.0
Belgium	1310.1250000000002
Finland	968.3975
Italy	934.995
Norway	0.0
Spain	1448.69
Denmark	2121.2275
Ireland	7337.485
Venezuela	4004.261
USA	17982.369499999997
Mexico	491.37249999999995
UK	1645.1999999999998
Switzerland	1226.8410000000001
Canada	5137.8099999999995
Brazil	8029.7585
Poland	0.0
Portugal	996.2874999999999

only showing top 20 rows

EJERCICIO

- Escribir un sentencia Spark SQL que muestre las ventas por país ordenadas de mayor a menor
  - Las ventas totales se deben mostrar sin decimales (redondeadas)
  - Mostrar las 10 primeras
  - ORDER BY | ROUND

```
scala> result.show(10)
+-----------+------------+
|ShipCountry|ProductSales|
+-----------+------------+
|        USA|     17982.0|
|    Germany|     14356.0|
|    Austria|     11493.0|
|     Brazil|      8030.0|
|    Ireland|      7337.0|
|     Canada|      5138.0|
|     Sweden|      5029.0|
|     France|      4140.0|
|  Venezuela|      4004.0|
|    Denmark|      2121.0|
+-----------+------------+
only showing top 10 rows
```

EJERCICIO: SOLUCIÓN

```
val result = spark.sql("SELECT ShipCountry,
ROUND(SUM(OrderDetailsTable.UnitPrice * Qty * Discount))
AS ProductSales FROM OrdersTable INNER JOIN
OrderDetailsTable ON OrdersTable.OrderID =
OrderDetailsTable.OrderID GROUP BY ShipCountry ORDER
BY ProductSales DESC")

result.show(10)
```